intercepts the request and returns the expected response within the Pact interaction build phase.

The test function contains the HTTP request and any assertions to check that the request has happened correctly, as shown in listing 5.15. The client is set up with the mock server URL, and, as shown earlier in listing 5.7, the function needs to overwrite the base URL for Pact to work. The response blocks the thread until its completion of the client call to get movies. The client method `getMovies` calls the `"/movies"` URL, as shown earlier in listing 5.6. The test asserts that the response data returned isn't empty, confirming the request has been mocked properly.

Listing 5.15 Assertion

```
import com.example.movie.data.di.AppModule.MovieClient
import org.junit.jupiter.api.Assertions.assertTrue

fun `should get movies response`(mockServer: MockServer) {
    var client = MovieClient(mockServer.getUrl())
    val response = runBlocking {
      client.getMovies()
    }
    assertTrue(response.isNotEmpty())
}
```

5.3 Android consumer contract generation

Running the test in Android Studio, you should see the green arrow icon by the test class or test function, as shown in figure 5.7.

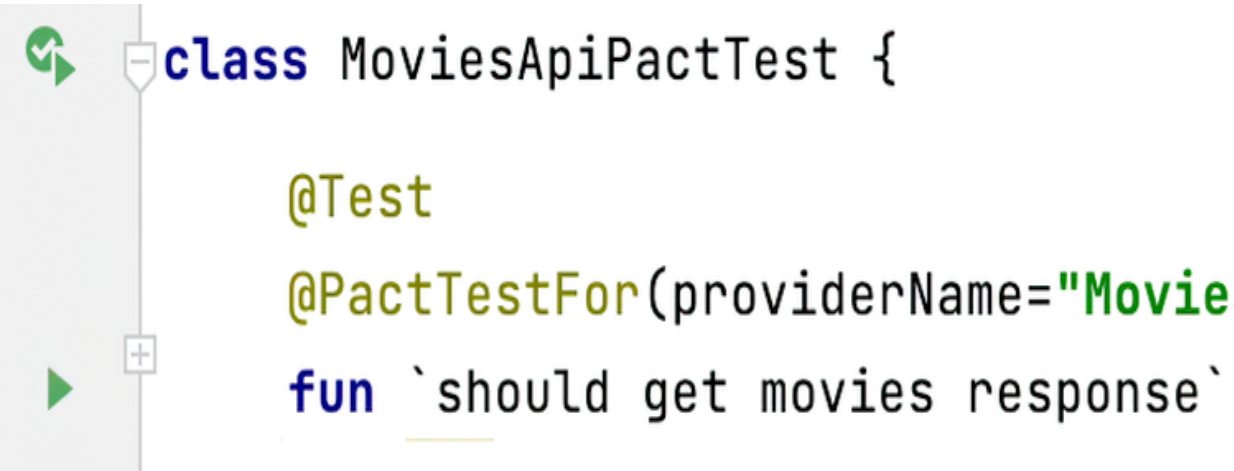


Figure 5.7 Test class and test function with green arrows in Android Studio margin on the left